АЛМАТИНСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ

Кафедра информационных систем

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №3

Проектирование связей и сложных структур данных в MongoDB

Дисциплина: «Проектирование и применение NoSQL-технологий»

3 курс, специальность «Информационные системы»

Вариант 5. Онлайн-библиотека (User, Book, Author, Loan, Review)

Студент: _____ Группа: _____

Преподаватель: _____

Алматы, 2026

1. Цель работы

Получить навыки проектирования сложных документов и связей в MongoDB: вложенные документы, массивы, dot notation, операторы $push, $pull, $addToSet, $elemMatch, связи 1:1, 1:N и N:M, Embedding и Referencing, Cardinality, Unbounded Arrays, денормализация и Snapshot.

2. Вариант и предметная область

Вариант 5 — «Онлайн-библиотека». Сервис хранит читателей, авторов, книги, выдачи и отзывы. База данных: `library_lab3`, шесть коллекций, 41 начальный документ (в ходе задач добавляются ещё 4: выдача 9011 и три события).

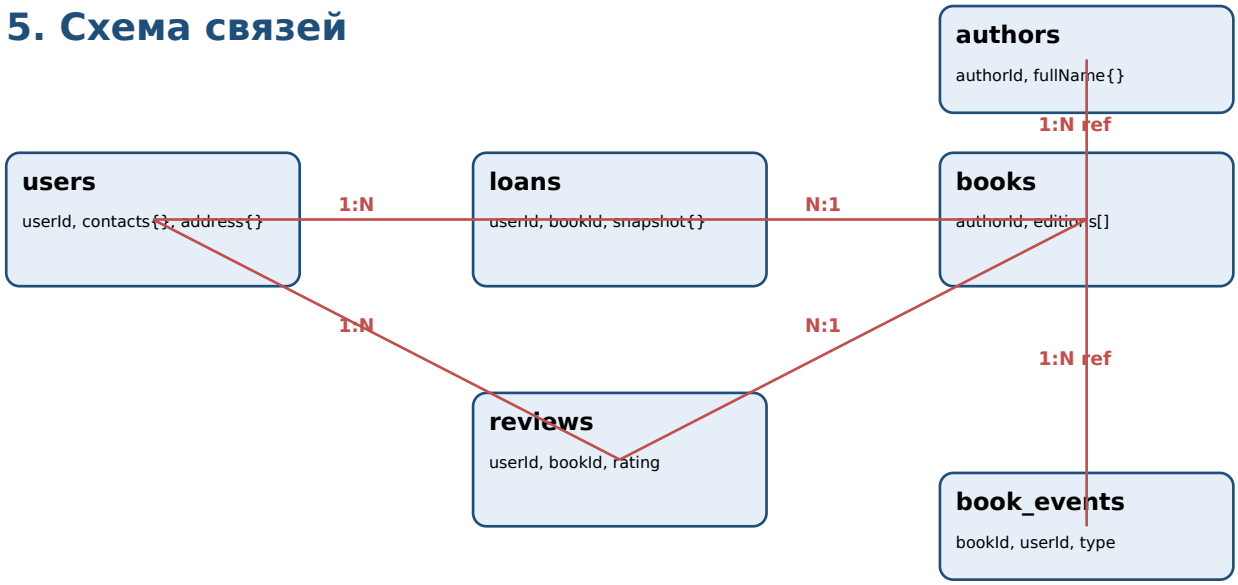
3. Сущности и атрибуты

Сущность / коллекция	Ключевые атрибуты	Документов
User (users)	userId, fullName{firstName,lastName}, contacts{email,phone}, address{city,street,building}, membership, interests[], registeredAt, active	6
Author (authors)	authorId, fullName{}, country, birthYear	5
Book (books)	bookId, title, authorId (ссылка), authorName (копия), genres[], year, copiesAvailable, ratingAvg, editions[] (массив документов)	8
Loan (loans)	loanId, userId, bookId, loanDate, dueDate, returnDate, status, fine, userSnapshot{}, bookSnapshot{}	10
Review (reviews)	reviewId, bookId, userId, rating, text, createdAt	6
BookEvent (book_events)	eventId, bookId, userId, eventType, createdAt	6

4. Access Patterns (12)

AP	Сценарий	Коллекция и фильтр	Тип / частота
AP-01	Профиль читателя	users {userId}	чтение, очень часто
AP-02	Читатели города	users {"address.city"}	чтение, средне
AP-03	Книги автора	books {authorId}	чтение, часто
AP-04	Поиск книг по жанру	books {genres: {$in}}	чтение, часто
AP-05	Книги по диапазону лет	books {year: {$gte,$lte}}	чтение, средне
AP-06	Топ книг по рейтингу	books, sort ratingAvg, limit	чтение, часто
AP-07	Издания книги	books {bookId}, поле editions	чтение, средне
AP-08	Оформить выдачу	loans insert + снимки читателя и книги	запись, часто
AP-09	Активные выдачи читателя	loans {userId, status}	чтение, очень часто
AP-10	Просроченные выдачи и штрафы	loans {$or: status, fine}	чтение, средне
AP-11	Отзывы о книге	reviews {bookId}	чтение, часто
AP-12	Фиксация просмотров и скачиваний	book_events insert	запись, постоянно

5. Схема связей



N:M User-Book реализовано отдельной сущностью loans (и reviews). 1:1 User-contacts/address - embedding.
Book 1:N editions - embedding (массив документов).

Рисунок 1. Схема коллекций и связей.

6. Embedding или Referencing: обоснование

Связь	Решение	Почему
User — contacts, address, preferences (1:1)	Embedded Document	Данные читаются вместе с читателем (AP-01), принадлежат только ему, размер ограничен, отдельный жизненный цикл не нужен.
Book — editions (1:N, N до ~10)	Embedded array of documents	Издания не живут без книги, читаются вместе с ней (AP-07), число ограничено — Bounded Array.
Author — Book (1:N)	Referencing (books.authorId)	Книга — самостоятельная сущность с собственными запросами; у автора десятки книг; встраивание дублировало бы данные автора.
Book — Review (1:N, тысячи)	Referencing (reviews.bookId)	Отзывов много и они растут; у отзыва свой жизненный цикл; вложенный массив стал бы Unbounded.
User — Book (N:M)	Отдельная коллекция loans	Связь имеет собственные атрибуты (даты, статус, штраф), поэтому это самостоятельная сущность Enrollment-типа.
Book — events (1:N, миллионы)	Отдельная коллекция book_events	События бесконечно накапливаются; массив внутри книги раздувает документ (лимит 16 МБ).

7. Cardinality, Bounded и Unbounded Array

Поле / связь	Оценка N	Класс	Вывод
users.interests	до 20	Bounded	встроенный массив простых значений
books.genres	до 5	Bounded	встроенный массив простых значений
books.editions	до 10	Bounded	встроенный массив документов
Author — Book	десятки	One-to-Many	ссылка в книге
Book — Review	тысячи	One-to-Squillions	ссылка в отзыве
Book — events / activityLog	миллионы	Unbounded	отдельная коллекция (задача 25)

8. Денормализация и Snapshot

Денормализация. В книге хранится копия `authorName`: вывод каталога не требует $lookup в authors. Цена: при переименовании автора нужно обновить его книги (updateMany по authorId); оправдано, потому что имена меняются очень редко, а каталог читается постоянно.

Snapshot. Выдача хранит `userSnapshot` (ФИО, email, тип членства) и `bookSnapshot` (название, автор) на момент выдачи вместе со ссылками userId и bookId. Если читатель сменит email или книга будет переименована, исторические выдачи останутся корректными (задача 24). Ссылка нужна для актуальных данных, снимок — для истории.

9. Коллекции и примеры документов

db.users

```
{userId: 1001, fullName: {firstName: "Aidar", lastName: "Serikov"}, contacts: {email: "aidar@example.com",
phone: "+7 700 111 22 33"}, address: {city: "Almaty", street: "Abay", building: 10}, membership: "standard",
interests: [], registeredAt: ISODate("2025-09-01"), active: true}
```

db.authors

```
{authorId: 305, fullName: {firstName: "Isaac", lastName: "Asimov"}, country: "USA", birthYear: 1920}
```

db.books

```
{bookId: 105, title: "Crime and Punishment", authorId: 304, authorName: "Fyodor Dostoevsky", genres: ["novel", "
classic", "psychology"], year: 1866, copiesAvailable: 6, ratingAvg: 4.9, editions: [{year: 2002, publisher: "
Penguin", pages: 576}, ...]}
```

db.loans

```
{loanId: 9001, userId: 1001, bookId: 105, loanDate: ISODate("2026-10-01"), dueDate: ISODate("2026-10-15"),
returnDate: ISODate("2026-10-12"), status: "returned", fine: 0, userSnapshot: {fullName: "Aidar Serikov",
email: "aidar@example.com", membership: "standard"}, bookSnapshot: {title: "Crime and Punishment", authorName: "
Fyodor Dostoevsky"}}
```

db.reviews

```
{reviewId: 7001, bookId: 105, userId: 1001, rating: 5, text: "Powerful psychological novel", createdAt:
ISODate("2026-10-13")}
```

db.book_events

```
{eventId: 50001, bookId: 107, userId: 1001, eventType: "view", createdAt: ISODate("2026-10-19T18:00:00Z")}
```

Полные данные (41 документ) находятся в файле lab3_variant5.js.

10. Практические задачи 1–25

Скрипт lab3_variant5.js запускается в mongosh (mongosh --file lab3_variant5.js). Вывод ниже получен выполнением тех же операций в среде эмуляции MongoDB (mongomock); значения _id опущены проекцией.

Задача 1. Создать БД

```
use library_lab3
```

```
switched to db library_lab3
```

Задача 2. Создать минимум 4 коллекции (создано 6)

```
db.createCollection("users")
db.createCollection("authors")
db.createCollection("books")
db.createCollection("loans")
db.createCollection("reviews")
db.createCollection("book_events")
```

```
{ ok: 1 }
{ ok: 1 }
{ ok: 1 }
{ ok: 1 }
{ ok: 1 }
{ ok: 1 }
```

Задача 3. Добавить минимум 25 документов (добавлено 41)

```
db.users.insertMany([... документы из lab3_variant5.js ...])
db.authors.insertMany([... документы из lab3_variant5.js ...])
db.books.insertMany([... документы из lab3_variant5.js ...])
db.loans.insertMany([... документы из lab3_variant5.js ...])
db.reviews.insertMany([... документы из lab3_variant5.js ...])
db.book_events.insertMany([... документы из lab3_variant5.js ...])
db.users.countDocuments()
db.authors.countDocuments()
db.books.countDocuments()
db.loans.countDocuments()
db.reviews.countDocuments()
db.book_events.countDocuments()
```

```
users: inserted 6
authors: inserted 5
books: inserted 8
loans: inserted 10
reviews: inserted 6
book_events: inserted 6
users=6, authors=5, books=8, loans=10, reviews=6, book_events=6
```

Задача 4. Создать Embedded Document (preferences во вложенном документе)

```
db.users.updateOne({"userId": 1001}, {"$set": {"preferences": {"language": "kk", "notify": {"email": true, "
sms": false}}}})
db.users.find({"userId": 1001}, {"_id": 0, "userId": 1, "preferences": 1})
```

```
{ matchedCount: 1, modifiedCount: 1 }
{"userId": 1001, "preferences": {"language": "kk", "notify": {"email": true, "sms": false}}}
```

Вложенный документ preferences создан одним $set; доступ к его полям — через точку (preferences.notify.email).

Задача 5. Создать массив простых значений (interests)

```
db.users.updateOne({"userId": 1001}, {"$set": {"interests": ["classic", "sci-fi", "history"]}})
db.users.find({"userId": 1001}, {"_id": 0, "userId": 1, "interests": 1})
```

```
{ matchedCount: 1, modifiedCount: 1 }
{"userId": 1001, "interests": ["classic", "sci-fi", "history"]}
```

Массив простых значений: порядок и дубли допустимы, пока не применён $addToSet.

Задача 6. Создать массив документов (editions у книг 105 и 103)

```
db.books.updateOne({"bookId": 105}, {"$set": {"editions": [{"year": 2002, "publisher": "Penguin", "pages":
576}, {"year": 2015, "publisher": "Vintage", "pages": 545}]}})
db.books.updateOne({"bookId": 103}, {"$set": {"editions": [{"year": 1983, "publisher": "Indiana University
Press", "pages": 352}, {"year": 2008, "publisher": "Penguin", "pages": 336}]}})
db.books.find({"editions": {"$exists": true}}, {"_id": 0, "bookId": 1, "editions": 1})
```

```
{ matchedCount: 1, modifiedCount: 1 }
{ matchedCount: 1, modifiedCount: 1 }
{"bookId": 103, "editions": [{"year": 1983, "publisher": "Indiana University Press", "pages": 352}, {"year":
2008, "publisher": "Penguin", "pages": 336}]}
{"bookId": 105, "editions": [{"year": 2002, "publisher": "Penguin", "pages": 576}, {"year": 2015, "publisher": "
Vintage", "pages": 545}]}
```

Задача 7. Поиск через dot notation

```
db.users.find({"address.city": "Almaty"}, {"_id": 0, "userId": 1, "fullName.firstName": 1, "contacts.email":
1})
```

```
{"userId": 1001, "fullName": {"firstName": "Aidar"}, "contacts": {"email": "aidar@example.com"}}
{"userId": 1003, "fullName": {"firstName": "Dana"}, "contacts": {"email": "dana@example.com"}}
{"userId": 1005, "fullName": {"firstName": "Zarina"}, "contacts": {"email": "zarina@example.com"}}
```

Задача 8. Использовать $push

```
db.users.updateOne({"userId": 1001}, {"$push": {"interests": "poetry"}})
db.users.find({"userId": 1001}, {"_id": 0, "interests": 1})
```

```
{ matchedCount: 1, modifiedCount: 1 }
{"interests": ["classic", "sci-fi", "history", "poetry"]}
```

$push добавляет элемент в конец массива всегда, даже если такое значение уже есть.

Задача 9. Использовать $pull

```
db.users.updateOne({"userId": 1001}, {"$pull": {"interests": "history"}})
db.users.find({"userId": 1001}, {"_id": 0, "interests": 1})
```

```
{ matchedCount: 1, modifiedCount: 1 }
{"interests": ["classic", "sci-fi", "poetry"]}
```

$pull удаляет из массива все элементы, равные значению или удовлетворяющие условию.

Задача 10. Использовать $addToSet (дубль не добавится, новое значение добавится)

```
db.users.updateOne({"userId": 1001}, {"$addToSet": {"interests": "poetry"}})
db.users.updateOne({"userId": 1001}, {"$addToSet": {"interests": "fantasy"}})
db.users.find({"userId": 1001}, {"_id": 0, "interests": 1})
```

```
{ matchedCount: 1, modifiedCount: 0 }
{ matchedCount: 1, modifiedCount: 1 }
{"interests": ["classic", "sci-fi", "poetry", "fantasy"]}
```

$addToSet добавляет значение, только если его ещё нет: "poetry" уже был (modifiedCount 0), "fantasy" добавлен. Массив interests ограничен по размеру, поэтому это Bounded Array.

Задача 11. Использовать $elemMatch

```
db.books.find({"editions": {"$elemMatch": {"publisher": "Penguin", "year": {"$gte": 2000}}}}, {"_id": 0, "
bookId": 1, "title": 1})
```

```
{"bookId": 103, "title": "The Day Lasts More Than a Hundred Years"}
{"bookId": 105, "title": "Crime and Punishment"}
```

$elemMatch требует, чтобы одно издание одновременно удовлетворяло обоим условиям (Penguin и год от 2000).

Задача 12. Реализовать связь 1:1 (User - contacts/address через embedding)

```
db.users.find({"contacts.email": "aidar@example.com"}, {"_id": 0, "userId": 1, "fullName": 1, "contacts": 1, "
address": 1})
```

```
{"userId": 1001, "fullName": {"firstName": "Aidar", "lastName": "Serikov"}, "contacts": {"email": "
aidar@example.com", "phone": "+7 700 111 22 33"}, "address": {"city": "Almaty", "street": "Abay", "building":
10}}
```

Связь 1:1: контакты и адрес лежат внутри документа читателя, отдельного join не нужно.

Задача 13. Реализовать 1:N через embedding (Book - editions)

```
db.books.find({"bookId": 105}, {"_id": 0, "title": 1, "editions": 1})
```

```
{"title": "Crime and Punishment", "editions": [{"year": 2002, "publisher": "Penguin", "pages": 576}, {"year":
2015, "publisher": "Vintage", "pages": 545}]}
```

Связь 1:N через embedding: издания читаются вместе с книгой одним запросом.

Задача 14. Реализовать 1:N через referencing (Author - Book по authorId)

```
db.books.find({"authorId": 305}, {"_id": 0, "bookId": 1, "title": 1, "authorId": 1})
```

```
{"bookId": 107, "title": "Foundation", "authorId": 305}
{"bookId": 108, "title": "I, Robot", "authorId": 305}
```

Связь 1:N через referencing: книга хранит authorId, автор не хранит списка книг.

Задача 15. Реализовать N:M (новая выдача связывает User и Book)

```
db.loans.insertOne({"loanId": 9011, "userId": 1006, "bookId": 107, "loanDate":
ISODate("2026-11-08T00:00:00Z"), "dueDate": ISODate("2026-11-22T00:00:00Z"), "returnDate": null, "status": "
active", "fine": 0, "userSnapshot": {"fullName": "Timur Alimov", "email": "timur@example.com", "membership": "
standard"}, "bookSnapshot": {"title": "Foundation", "authorName": "Isaac Asimov"}})
db.loans.find({"loanId": 9011}, {"_id": 0, "loanId": 1, "userId": 1, "bookId": 1, "status": 1})
```

```
{ acknowledged: true }
{"loanId": 9011, "userId": 1006, "bookId": 107, "status": "active"}
```

N:M реализована сущностью Loan: документ выдачи связывает читателя и книгу и несёт собственные атрибуты.

Задача 16. Выполнить запрос по ссылке ($lookup: автор вместе со своими книгами)

```
db.authors.aggregate([{"$match": {"authorId": 303}}, {"$lookup": {"from": "books", "localField": "authorId", "
foreignField": "authorId", "as": "books"}}, {"$project": {"_id": 0, "fullName": 1, "books.title": 1}}])
```

```
{"fullName": {"firstName": "Chinghiz", "lastName": "Aitmatov"}, "books": [{"title": "The Day Lasts More Than a
Hundred Years"}, {"title": "Jamila"}]}
```

$lookup соединяет authors и books по authorId: полный документ автора не встроен в каждую книгу.

Задача 17. Запрос к промежуточной коллекции (активные выдачи читателя)

```
db.loans.find({"userId": 1001, "status": "active"}, {"_id": 0, "loanId": 1, "bookId": 1, "dueDate": 1})
```

```
{"loanId": 9002, "bookId": 107, "dueDate": "2026-11-03"}
```

Задача 18. Использовать $gte/$lte

```
db.books.find({"year": {"$gte": 1900, "$lte": 1999}}, {"_id": 0, "title": 1, "year": 1})
```

```
{"title": "Book of Words", "year": 1904}
{"title": "Abai Zholy", "year": 1947}
{"title": "The Day Lasts More Than a Hundred Years", "year": 1980}
{"title": "Jamila", "year": 1958}
{"title": "Foundation", "year": 1951}
{"title": "I, Robot", "year": 1950}
```

Задача 19. Использовать $in

```
db.books.find({"genres": {"$in": ["sci-fi", "psychology"]}}, {"_id": 0, "bookId": 1, "title": 1})
```

```
{"bookId": 103, "title": "The Day Lasts More Than a Hundred Years"}
{"bookId": 105, "title": "Crime and Punishment"}
{"bookId": 107, "title": "Foundation"}
{"bookId": 108, "title": "I, Robot"}
```

Задача 20. Использовать $or

```
db.loans.find({"$or": [{"status": "overdue"}, {"fine": {"$gt": 0}}]}, {"_id": 0, "loanId": 1, "userId": 1, "
status": 1, "fine": 1})
```

```
{"loanId": 9004, "userId": 1002, "status": "overdue", "fine": 1500}
{"loanId": 9005, "userId": 1003, "status": "returned", "fine": 600}
{"loanId": 9006, "userId": 1004, "status": "overdue", "fine": 300}
```

Задача 21. Сортировка (по рейтингу, убывание)

```
db.books.find({}, {"_id": 0, "title": 1, "ratingAvg": 1}).sort({"ratingAvg": -1})
```

```
{"title": "Crime and Punishment", "ratingAvg": 4.9}
{"title": "Book of Words", "ratingAvg": 4.8}
{"title": "Abai Zholy", "ratingAvg": 4.7}
{"title": "Jamila", "ratingAvg": 4.6}
{"title": "Foundation", "ratingAvg": 4.6}
{"title": "The Day Lasts More Than a Hundred Years", "ratingAvg": 4.5}
{"title": "The Idiot", "ratingAvg": 4.4}
{"title": "I, Robot", "ratingAvg": 4.3}
```

Задача 22. Проекция

```
db.users.find({}, {"_id": 0, "userId": 1, "fullName.lastName": 1, "contacts.email": 1})
```

```
{"userId": 1001, "fullName": {"lastName": "Serikov"}, "contacts": {"email": "aidar@example.com"}}
{"userId": 1002, "fullName": {"lastName": "Nurova"}, "contacts": {"email": "aliya@example.com"}}
{"userId": 1003, "fullName": {"lastName": "Omarova"}, "contacts": {"email": "dana@example.com"}}
{"userId": 1004, "fullName": {"lastName": "Kenesov"}, "contacts": {"email": "marat@example.com"}}
{"userId": 1005, "fullName": {"lastName": "Bekova"}, "contacts": {"email": "zarina@example.com"}}
{"userId": 1006, "fullName": {"lastName": "Alimov"}, "contacts": {"email": "timur@example.com"}}
```

Задача 23. Использовать limit() (топ-3 книги)

```
db.books.find({}, {"_id": 0, "title": 1, "ratingAvg": 1}).sort({"ratingAvg": -1}).limit(3)
```

```
{"title": "Crime and Punishment", "ratingAvg": 4.9}
{"title": "Book of Words", "ratingAvg": 4.8}
{"title": "Abai Zholy", "ratingAvg": 4.7}
```

Задача 24. Продемонстрировать денормализацию и Snapshot (смена email не меняет снимок в выдаче)

```
db.users.updateOne({"userId": 1001}, {"$set": {"contacts.email": "aidar.new@example.com"}})
db.users.find({"userId": 1001}, {"_id": 0, "userId": 1, "contacts.email": 1})
db.loans.find({"loanId": 9001}, {"_id": 0, "loanId": 1, "userSnapshot": 1, "bookSnapshot": 1})
```

```
{ matchedCount: 1, modifiedCount: 1 }
{"userId": 1001, "contacts": {"email": "aidar.new@example.com"}}
{"loanId": 9001, "userSnapshot": {"fullName": "Aidar Serikov", "email": "aidar@example.com", "membership": "
standard"}, "bookSnapshot": {"title": "Crime and Punishment", "authorName": "Fyodor Dostoevsky"}}
```

Email читателя изменён, но userSnapshot в выдаче 9001 сохранил значение на момент выдачи.

Задача 25. Найти и исправить Unbounded Array (activityLog -> коллекция book_events)

```
db.books.find({"activityLog": {"$exists": true}}, {"_id": 0, "bookId": 1, "activityLog": 1})
// перенос записей activityLog в отдельную коллекцию
db.book_events.insertMany([
 {eventId: 50007, bookId: 105, userId: 1001, eventType: "view", createdAt: ISODate("2026-10-01T09:00:00Z")},
 {eventId: 50008, bookId: 105, userId: 1002, eventType: "view", createdAt: ISODate("2026-10-01T10:30:00Z")},
 {eventId: 50009, bookId: 105, userId: 1003, eventType: "download", createdAt: ISODate("2026-10-02T12:15:00Z")}
])
db.books.updateOne({"bookId": 105}, {"$unset": {"activityLog": ""}})
db.book_events.find({"bookId": 105}, {"_id": 0, "eventId": 1, "userId": 1, "eventType": 1})
db.books.find({"bookId": 105}, {"_id": 0, "bookId": 1, "activityLog": 1})
```

```
{"bookId": 105, "activityLog": [{"userId": 1001, "eventType": "view", "createdAt": "2026-10-01 09:00"},
{"userId": 1002, "eventType": "view", "createdAt": "2026-10-01 10:30"}, {"userId": 1003, "eventType": "
download", "createdAt": "2026-10-02 12:15"}]}
{ acknowledged: true, insertedCount: 3 }
{ matchedCount: 1, modifiedCount: 1 }
{"eventId": 50007, "userId": 1001, "eventType": "view"}
{"eventId": 50008, "userId": 1002, "eventType": "view"}
{"eventId": 50009, "userId": 1003, "eventType": "download"}
{"bookId": 105}
```

Проблема: activityLog внутри книги растёт без границы и приближает документ к лимиту 16 МБ. Решение: события перенесены в book_events, а поле удалено ($unset).

11. Архитектурная схема и вывод

Схема коллекций приведена на рисунке 1. Модель библиотеки строилась от Access Patterns: данные, которые читаются вместе и ограничены по размеру (контакты, адрес, издания), встроены; растущие и самостоятельные сущности (отзывы, события, выдачи) вынесены в отдельные коллекции со ссылками.

Выполнены все 25 задач: создана база и 6 коллекций, добавлено 41 документ, реализованы связи 1:1, 1:N (embedding и referencing) и N:M (через loans), использованы $push, $pull, $addToSet, $elemMatch, $gte/$lte, $in, $or, сортировка, проекция, limit, денормализация, Snapshot, найден и исправлен Unbounded Array.

Компромиссы: встраивание ускоряет чтение, но усложняет независимые обновления; ссылки гибки, но требуют второго запроса или $lookup; снимки занимают место, зато сохраняют историю. Риск: authorName дублируется и при переименовании автора должен обновляться вместе с книгами.

12. Ответы на контрольные вопросы

1. Что такое Embedded Document? Документ, вложенный как значение поля в другой документ (например, address в users).

2. Что такое dot notation? Запись пути к вложенному полю через точку: "address.city", "contacts.email", "editions.year".

3. Чем $addToSet отличается от $push? $push добавляет элемент всегда, $addToSet — только если такого значения в массиве ещё нет.

4. Для чего применяется $pull? Удаляет из массива элементы, совпадающие со значением или условием.

5. Что делает $elemMatch? Находит документы, у которых хотя бы один элемент массива одновременно удовлетворяет всем перечисленным условиям.

6. Что называется кардинальностью? Число связанных объектов на каждой стороне связи (и оценка N): один, десятки, тысячи, миллионы.

7. Что означают 1:1, 1:N и N:M? Один к одному, один ко многим и многие ко многим: читатель — свой адрес; автор — его книги; читатели — книги через выдачи.

8. Что такое embedding и referencing? Embedding — хранение связанных данных внутри документа; referencing — хранение идентификатора и выборка связанного документа отдельно.

9. Что такое Bounded и Unbounded Array? Bounded — массив с известной небольшой верхней границей (genres, editions); Unbounded — массив, растущий без предела (activityLog), его выносят в коллекцию.

10. Почему при 1:N важно знать реальное N? От N зависит выбор: небольшое ограниченное N можно встроить, тысячи и больше — нужна ссылка или отдельная коллекция.

11. Что такое денормализация? Намеренное дублирование данных (authorName в книге) ради скорости чтения ценой усложнения обновлений.

12. Что такое Snapshot? Копия данных на момент события (userSnapshot в выдаче), которая не меняется при изменении оригинала.

13. Почему жизненный цикл объекта влияет на модель? Объекты с разным временем жизни и частотой изменений нельзя встраивать друг в друга: отзывы переживают правки книги, выдачи — смену email.

14. Когда N:M целесообразно представить отдельной коллекцией? Когда связь имеет собственные атрибуты (даты, статус, штраф) или быстро растёт и запрашивается сама по себе.

15. Как Access Patterns влияют на модель? Структура документов подбирается под частые запросы: что читается вместе, то хранится вместе, а редкие или тяжёлые сценарии получают отдельные коллекции.